DOCUMENT 07 / SMART UNIVERSITY DIGITAL CAMPUS

Technology Architecture

Technical approach, data flow, scalability, and security for an evaluable campus platform.

Architecture assessment The repository implements a useful modular application foundation. The deployed production perimeter, high availability, full audit UI, secure device token store, and large-scale throughput have not been demonstrated.

Current system path

Layer	Current choice	Reason and limit
Client	Flutter for student, parent, and admin portals; Windows demonstration.	Shared UI code and desktop flow; test Android/iOS/web separately.
API	FastAPI REST with JWT role checks and Pydantic schemas.	Readable modular endpoints; every record needs server-side ownership checks.
Persistence	SQLAlchemy; SQLite locally; PostgreSQL 18 in Compose example.	Structured relational model; production sizing and migrations need validation.
Documents	ReportLab PDF and QR verification for certificates.	Issuance and revocation flow; public URL/domain and minimum disclosure need review.
Assistance	Backend-held Gemini key, local context and knowledge routing.	Provider is optional for open questions; quality and data controls require evaluation.
Risk	Weighted attendance/academic/fee/engagement score.	Human-review signal, not a trained or calibrated probability.

Logical architecture

User surface	Application boundary	Data / external boundary
Student • Parent • Admin Flutter	HTTPS edge (proposed) FastAPI auth + workflow APIs	PostgreSQL (production target) records, requests, certificates
Voice/chat UI	Context and policy routing AI provider adapter	Approved knowledge + Gemini API only when needed
Admin operations	Imports • PDF issuance risk review	Institutional source records backups + monitoring (proposed)

A modular application suits this stage: one backend contains well-bounded academic, service, identity, and AI modules, with a relational source of truth. Separate services are justified only when measured scaling or ownership requires them.

Authoritative data and workflow

University officers create and maintain programmes, courses, academic plans, rooms, routes, and certificate rules. Admin registration links a real student to a unique institutional identity. Academic and service transactions then update relational records; student and parent clients fetch authorized views. Parent access depends on an explicit stored link to the student.

Domain	Primary steward	Consistency rule
Identity + parent links	Registrar / accounts team	One account-to-person binding; relationship changes logged and reviewed.
Academic catalogue	Academic office	Approved programme/semester/course version; validate imported credit totals.
Attendance + examinations	Faculty / exam office	Source ownership and timestamp; late corrections traceable.
Fees	Finance office	Invoice, recorded payment, and adjustment reconcile; gateway is future work.
Hostel + transport	Service officers	Capacity and allocation status agree with available inventory.
Certificates	Authorized issuer	Request-to-issue-to-revoke state chain with minimal public verification data.

AI answer path

The assistant first tries authenticated student context for personal questions, then approved campus guidance and curated general answers. Open questions may go to Gemini via the backend. The service stores a short conversation context; previous assistant answers can contain personal data. Redaction or strict exclusion of such history is a production requirement before external forwarding.

English and Hindi choices and voice UI are present. This does not establish support for all Indian languages; Bhashini and AIKosh are not integrated in the verified code. Provider model availability, rate limits, quotas, and costs must be confirmed in the actual account before launch.

Predictive-risk claim

The current score uses weighted attendance (40%), academic results (30%), fees (15%), and engagement (15%). Thresholds classify low below 40, medium 40 to under 70, and high from 70. The API currently exposes a dropout_probability value derived from the risk score; it is **not** a calibrated probability. A trained model would need approved historical labels, time-separated validation, bias checks, and prospective monitoring.

Security: current controls and required work

Concern	Current evidence	Required before institutional pilot
Login and roles	HS256 JWT, 60-minute access token, bcrypt, active-user lookup.	Endpoint-by-endpoint role and object checks; staff MFA and token revocation policy.
Device session	Token stored with Flutter SharedPreferences.	Use OS-backed secure storage; test expiry, logout, and shared devices.
API secrets	Gemini key read by backend configuration.	Rotate exposed keys; secret manager, access review, no client delivery.
Transport	Local HTTP; production guide recommends reverse proxy with HTTPS.	TLS termination, secure headers, rate limiting, network policy.
AI privacy	Context routing and previous answers can enter provider prompt.	Minimize/redact prompts; consent and retention decision; provider contract review.
Certificate verification	QR verification and revocation route.	Production domain, issuance authority, minimal public fields, tamper tests.
Audit/recovery	Workflow state exists; full admin audit UI unavailable.	Central event logs, backups, restore drills, access and retention controls.

Object-level authorization is critical because students and parents access records addressed by IDs. Negative tests should try another student's invoices, attendance, allocations, and certificates using valid sessions. The UI's hidden controls are not an access boundary.

Security evaluation gate

Run a targeted review of authentication, broken object authorization, role escalation, import validation, PDF/QR disclosure, logging, and AI prompt handling. Close critical findings and retest before institutional records enter the pilot. A working demo is not a security certification.

OWASP API object authorization https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/

Scalability and reliability

The example deployment has one backend container with two Gunicorn workers and one PostgreSQL container. It is a deployment starting point, not high availability. SQLAlchemy enables pool pre-ping; connection counts, indexes, paging, and query cost require measurement with realistic institutional data.

Scale pressure	Planned response	Proof before expansion
Larger student rosters	Paginate lists; index joins and search fields; batch related lookups.	Representative seed volume and p95 query timing.
AI bursts / provider outage	Overall request budget, quotas, bounded retries, graceful local fallback.	Rate-limit and timeout fault tests; cost report.
PDF and bulk imports	Isolate longer jobs; use idempotent task identifiers and completion status.	Repeated-request and interrupted-job tests.
Multiple replicas	Serialized migrations; shared database; coordinated rolling releases.	Deployment/rollback exercise; no concurrent schema upgrade.
Data loss	Scheduled backups and tested restore; choose RPO/RTO with university.	Isolated restore and record checks.

Operational targets to validate

For a controlled pilot, propose ordinary read p95 under 2 seconds, under 1% server errors, and 99.5% availability during the agreed service window. A proposed initial recovery objective is RPO 24 hours and RTO 4 hours; examination and finance owners may require tighter objectives. These are planning targets, not measured service levels.

Technical decision

Proceed with one secured backend and PostgreSQL while the cohort is small. Measure the workload and add operational capacity only after a baseline. Keep the AI service behind the backend, distinguish human-reviewed risk signals from validated prediction, and validate every platform and integration before claiming campus-wide readiness.

Sources

Project: **backend/app/core/security.py**, **backend/app/core/database.py**, **backend/app/api/v1**, **backend/app/services**, **backend/Dockerfile**, **backend/docker-compose.production.yml**, and Flutter **lib**.

FastAPI deployment concepts https://fastapi.tiangolo.com/deployment/concepts/

PostgreSQL backup guidance https://www.postgresql.org/docs/18/backup.html

Gemini API limits https://ai.google.dev/gemini-api/docs/rate-limits